Backtracking & Approximation Algorithms

N-Queens, Sudoku Solver, Graph Coloring, Hamiltonian Cycle, Approximation Algorithms (Vertex Cover / Bin Packing)

Guide : Dr. Sriramya
Name: Jaasiel J S

N-Queens Problem

1. Place N Queens on a Chessboard

Basic | Backtracking

Place N queens on an N × N chessboard such that no two queens attack each other. Display one valid arrangement.

Input:

N = 4

Expected Output:

One Valid Solution:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

main.py

```
def solve_nqueens(n, all_solutions=False):
    cols=set(); diag1=set(); diag2=set(); pos=[-1]*n; out=[]
    def bt(r):
        if r==n:
            out.append(pos.copy()); return not all_solutions
        for c in range(n):
            if c in cols or r-c in diag1 or r+c in diag2: continue
            pos[r]=c; cols.add(c); diag1.add(r-c); diag2.add(r+c)
            if bt(r+1) and not all_solutions: return True
            cols.remove(c); diag1.remove(r-c); diag2.remove(r+c)
        return False
    bt(0); return out

def show(sol):
    for c in sol:
        print(' '.join('1' if j==c else '0' for j in range(len(sol))))

sol=solve_nqueens(4)[0]
print('One Valid Solution:')
show(sol)
```

Output

```
One Valid Solution:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
```

2. Find All Solutions of N-Queens

Medium | Backtracking

Find all possible ways to place N queens on an N × N chessboard.

Input:

N = 4

Expected Output:

Total Solutions = 2

Solution 1:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Solution 2:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

main.py

```
def solve_nqueens(n, all_solutions=False):
    cols=set(); diag1=set(); diag2=set(); pos=[-1]*n; out=[]
    def bt(r):
        if r==n:
            out.append(pos.copy()); return not all_solutions
        for c in range(n):
            if c in cols or r-c in diag1 or r+c in diag2: continue
            pos[r]=c; cols.add(c); diag1.add(r-c); diag2.add(r+c)
            if bt(r+1) and not all_solutions: return True
            cols.remove(c); diag1.remove(r-c); diag2.remove(r+c)
        return False
    bt(0); return out

def show(sol):
    for c in sol:
        print(' '.join('1' if j==c else '0' for j in range(len(sol))))

sols=solve_nqueens(4,True)
print('Total Solutions =',len(sols))
for i,s in enumerate(sols,1):
    print(f'Solution {i}:'); show(s)
```

Output

```
Total Solutions = 2
Solution 1:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
Solution 2:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```

3. Count Total N-Queens Solutions

Hard | Solution Counting

Determine the total number of valid solutions for the N-Queens problem.

Input:

N = 8

Expected Output:

Total Number of Solutions = 92

main.py

```
def solve_nqueens(n, all_solutions=False):
    cols=set(); diag1=set(); diag2=set(); pos=[-1]*n; out=[]
    def bt(r):
        if r==n:
            out.append(pos.copy()); return not all_solutions
        for c in range(n):
            if c in cols or r-c in diag1 or r+c in diag2: continue
            pos[r]=c; cols.add(c); diag1.add(r-c); diag2.add(r+c)
            if bt(r+1) and not all_solutions: return True
            cols.remove(c); diag1.remove(r-c); diag2.remove(r+c)
        return False
    bt(0); return out

def show(sol):
    for c in sol:
        print(' '.join('1' if j==c else '0' for j in range(len(sol))))

print('Total Number of Solutions =',len(solve_nqueens(8,True)))
```

Output

```
Total Number of Solutions = 92
```

Sudoku Solver

4. Validate Sudoku Placement

Basic | Constraint Checking

Check whether a given number can be placed safely in a specified Sudoku cell.

Input:

Cell = (0,2)
Number = 4

Expected Output:

Safe Placement: YES

main.py

```
grid=[
[5,3,0,0,7,0,0,0,0],
[6,0,0,1,9,5,0,0,0],
[0,9,8,0,0,0,0,6,0],
[8,0,0,0,6,0,0,0,3],
[4,0,0,8,0,3,0,0,1],
[7,0,0,0,2,0,0,0,6],
[0,6,0,0,0,0,2,8,0],
[0,0,0,4,1,9,0,0,5],
[0,0,0,0,8,0,0,7,9]]

def safe(g,r,c,num):
    return (num not in g[r] and
            all(g[i][c]!=num for i in range(9)) and
            all(g[i][j]!=num for i in range((r//3)*3,(r//3)*3+3)
                for j in range((c//3)*3,(c//3)*3+3)))
print('Safe Placement:', 'YES' if safe(grid,0,2,4) else 'NO')
```

Output

```
Safe Placement: YES
```

5. Solve a 9 × 9 Sudoku Puzzle

Medium | Backtracking

Solve the given Sudoku puzzle and display the completed grid.

Input:

Sudoku Grid:
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9

Expected Output:

Solved Sudoku Grid:
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9

main.py

```
grid=[
[5,3,0,0,7,0,0,0,0],
[6,0,0,1,9,5,0,0,0],
[0,9,8,0,0,0,0,6,0],
[8,0,0,0,6,0,0,0,3],
[4,0,0,8,0,3,0,0,1],
[7,0,0,0,2,0,0,0,6],
[0,6,0,0,0,0,2,8,0],
[0,0,0,4,1,9,0,0,5],
[0,0,0,0,8,0,0,7,9]]

def safe(g,r,c,num):
    if num in g[r]: return False
    if any(g[i][c]==num for i in range(9)): return False
    br=(r//3)*3; bc=(c//3)*3
    return all(g[i][j]!=num for i in range(br,br+3) for j in range(bc,bc+3))

def solve(g):
    for r in range(9):
        for c in range(9):
            if g[r][c]==0:
                for num in range(1,10):
                    if safe(g,r,c,num):
                        g[r][c]=num
                        if solve(g): return True
                        g[r][c]=0
                return False
    return True

solve(grid)
print('Solved Sudoku Grid:')
for row in grid: print(*row)
```

Output

```
Solved Sudoku Grid:
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

6. Determine Whether Sudoku is Solvable

Hard | Feasibility Check

Determine whether a valid solution exists for the given Sudoku puzzle.

Input:

Partially Filled Sudoku Grid

Expected Output:

Sudoku Solvable: YES
Solution Found Successfully

main.py

```
grid=[
[5,3,0,0,7,0,0,0,0],
[6,0,0,1,9,5,0,0,0],
[0,9,8,0,0,0,0,6,0],
[8,0,0,0,6,0,0,0,3],
[4,0,0,8,0,3,0,0,1],
[7,0,0,0,2,0,0,0,6],
[0,6,0,0,0,0,2,8,0],
[0,0,0,4,1,9,0,0,5],
[0,0,0,0,8,0,0,7,9]]

def safe(g,r,c,num):
    if num in g[r]: return False
    if any(g[i][c]==num for i in range(9)): return False
    br=(r//3)*3; bc=(c//3)*3
    return all(g[i][j]!=num for i in range(br,br+3) for j in range(bc,bc+3))

def solve(g):
    for r in range(9):
        for c in range(9):
            if g[r][c]==0:
                for num in range(1,10):
                    if safe(g,r,c,num):
                        g[r][c]=num
                        if solve(g): return True
                        g[r][c]=0
                return False
    return True

print('Sudoku Solvable:', 'YES' if solve(grid) else 'NO')
print('Solution Found Successfully' if all(all(v for v in row) for row in grid) else 'No Solution')
```

Output

```
Sudoku Solvable: YES
Solution Found Successfully
```

Graph Coloring

7. Color a Graph Using M Colors

Basic | Graph Coloring

Assign colors to vertices such that no adjacent vertices have the same color.

Input:

Vertices = 4
M = 3

Adjacency Matrix:
0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0

Expected Output:

Vertex 0 → Color 1
Vertex 1 → Color 2

Vertex 2 → Color 3
Vertex 3 → Color 2

main.py

```
def color_graph(n,edges,m):
    g=[set() for _ in range(n)]
    for u,v in edges: g[u].add(v); g[v].add(u)
    color=[0]*n
    def bt(v):
        if v==n:return True
        for c in range(1,m+1):
            if all(color[x]!=c for x in g[v]):
                color[v]=c
                if bt(v+1): return True
                color[v]=0
        return False
    return color if bt(0) else None

n=4; m=3
edges=[(0,1),(0,2),(0,3),(1,2),(2,3)]
colors=color_graph(n,edges,m)
for v,c in enumerate(colors): print(f'Vertex {v} -> Color {c}')
```

Output

```
Vertex 0 -> Color 1
Vertex 1 -> Color 2
Vertex 2 -> Color 3
Vertex 3 -> Color 2
```

8. Determine Whether M-Coloring is Possible

Medium | Feasibility Check

Determine whether the graph can be colored using M colors.

Input:

Vertices = 5
M = 3

Edges:
(0,1) (0,2) (1,2) (1,3) (2,4)

Expected Output:

Graph can be colored using 3 colors.

main.py

```
def color_graph(n,edges,m):
    g=[set() for _ in range(n)]
    for u,v in edges: g[u].add(v); g[v].add(u)
    color=[0]*n
    def bt(v):
        if v==n:return True
        for c in range(1,m+1):
            if all(color[x]!=c for x in g[v]):
                color[v]=c
                if bt(v+1): return True
                color[v]=0
        return False
    return color if bt(0) else None

n=5; m=3
edges=[(0,1),(0,2),(1,2),(1,3),(2,4)]
print('Graph can be colored using 3 colors.' if color_graph(n,edges,m) else 'Not possible')
```

Output

```
Graph can be colored using 3 colors.
```

9. Find Minimum Colors Required

Hard | Chromatic Number

Determine the minimum number of colors required to color the graph.

Input:

Complete Graph K4

Vertices = 4
Edges:
(0,1) (0,2) (0,3)
(1,2) (1,3)
(2,3)

Expected Output:

Minimum Colors Required = 4

main.py

```
def color_graph(n,edges,m):
    g=[set() for _ in range(n)]
    for u,v in edges: g[u].add(v); g[v].add(u)
    color=[0]*n
    def bt(v):
        if v==n:return True
        for c in range(1,m+1):
            if all(color[x]!=c for x in g[v]):
                color[v]=c
                if bt(v+1): return True
                color[v]=0
        return False
    return color if bt(0) else None

n=4
edges=[(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)]
for m in range(1,n+1):
    if color_graph(n,edges,m):
        print('Minimum Colors Required =',m); break
```

Output

```
Minimum Colors Required = 4
```

Hamiltonian Cycle

Q10. Detect Hamiltonian Cycle

Basic | Backtracking

Find a Hamiltonian Cycle in the graph if one exists.

Input:

Vertices = 5

Adjacency Matrix:
0 1 0 1 0
1 0 1 1 1
0 1 0 0 1
1 1 0 0 1
0 1 1 1 0

Expected Output:

Hamiltonian Cycle Exists:
0 → 1 → 2 → 4 → 3 → 0

main.py

```
def hamiltonian_path(n,edges,cycle=False):
    g=[set() for _ in range(n)]
    for u,v in edges: g[u].add(v); g[v].add(u)
    path=[0]; used={0}
    def bt():
        if len(path)==n:
            return (not cycle) or (path[0] in g[path[-1]])
        for v in sorted(g[path[-1]]):
            if v not in used:
                used.add(v); path.append(v)
                if bt(): return True
                path.pop(); used.remove(v)
        return False
    return path.copy() if bt() else None

n=5
edges=[(0,1),(0,3),(1,2),(1,3),(1,4),(2,4),(3,4)]
p=hamiltonian_path(n,edges,True)
print('Hamiltonian Cycle Exists:')
print(' -> '.join(map(str,p+[p[0]])))
```

Output

```
Hamiltonian Cycle Exists:
0 -> 1 -> 2 -> 4 -> 3 -> 0
```

11. Find Hamiltonian Path

Medium | Path Construction

Find a path that visits every vertex exactly once.

Input:

Vertices = 5

Edges:
(0,1) (1,2) (2,3) (3,4)

Expected Output:

Hamiltonian Path:
0 → 1 → 2 → 3 → 4

main.py

```python
def hamiltonian_path(n,edges,cycle=False):
    g=[set() for _ in range(n)]
    for u,v in edges: g[u].add(v); g[v].add(u)
    path=[0]; used={0}
    def bt():
        if len(path)==n:
            return (not cycle) or (path[0] in g[path[-1]])
        for v in sorted(g[path[-1]]):
            if v not in used:
                used.add(v); path.append(v)
                if bt(): return True
                path.pop(); used.remove(v)
        return False
    return path.copy() if bt() else None

n=5
edges=[(0,1),(1,2),(2,3),(3,4)]
p=hamiltonian_path(n,edges,False)
print('Hamiltonian Path:')
print(' -> '.join(map(str,p)))
```

Output

```
Hamiltonian Path:
0 -> 1 -> 2 -> 3 -> 4
```

12. Find All Hamiltonian Cycles

Hard | Exhaustive Search

Generate all distinct Hamiltonian cycles in the graph.

Input:

Complete Graph K4

Expected Output:

Hamiltonian Cycles:
0 → 1 → 2 → 3 → 0
0 → 1 → 3 → 2 → 0
0 → 2 → 1 → 3 → 0
Total Cycles = 3

main.py

```python
import itertools
n=4
cycles=[]
for perm in itertools.permutations(range(1,n)):
    # Complete graph: every permutation gives a cycle. Treat reverses as identical.
    if perm <= perm[::-1]: cycles.append((0,)+perm+(0,))
print('Hamiltonian Cycles:')
for c in cycles: print(' -> '.join(map(str,c)))
print('Total Cycles =',len(cycles))
```

Output

```
Hamiltonian Cycles:
0 -> 1 -> 2 -> 3 -> 0
0 -> 1 -> 3 -> 2 -> 0
0 -> 2 -> 1 -> 3 -> 0
Total Cycles = 3
```

Approximation Algorithm – Vertex Cover

Q13. Approximate Vertex Cover

Medium | Greedy Approximation

Find an approximate vertex cover using a greedy approach.

Input:

Vertices = 5

Edges:
(0,1)
(0,2)
(1,3)
(2,4)

Expected Output:

Approximate Vertex Cover:
{0,1,2}

Cover Size = 3

main.py

```
vertices=5
edges=[(0,1),(0,2),(1,3),(2,4)]
remaining=set(tuple(sorted(e)) for e in edges); cover=[]
while remaining:
    deg=[sum(v in e for e in remaining) for v in range(vertices)]
    v=max(range(vertices), key=lambda x:(deg[x],-x))
    cover.append(v)
    remaining={e for e in remaining if v not in e}
print('Approximate Vertex Cover:')
print(set(cover))
print('Cover Size =',len(cover))
```

Output

```
Approximate Vertex Cover:
{0, 1, 2}
Cover Size = 3
```

14. Compare Exact and Approximate Vertex Cover

Hard | Approximation Analysis

Compute both exact and approximate vertex covers.

Input:

Vertices = 6

Edges:
(0,1)
(0,2)

(1,3)
(2,4)
(3,5)

Expected Output:

Exact Vertex Cover Size = 3
Approximate Vertex Cover Size = 4
Approximation Ratio = 1.33

main.py

```
import itertools
vertices=6
edges=[(0,1),(0,2),(1,3),(2,4),(3,5)]
def is_cover(S): return all(u in S or v in S for u,v in edges)
exact=None
for r in range(vertices+1):
    for comb in itertools.combinations(range(vertices),r):
        if is_cover(set(comb)): exact=set(comb); break
    if exact is not None: break
# 2-approximation using a deterministic maximal matching.
# Choose high-degree uncovered edges first to reduce the cover size.
deg=[0]*vertices
for u,v in edges: deg[u]+=1; deg[v]+=1
remaining=set(edges); approx=set()
while remaining:
    u,v=max(remaining,key=lambda e:(deg[e[0]]+deg[e[1]],e[0]+e[1],e))
    approx.update([u,v])
    remaining={e for e in remaining if u not in e and v not in e}
print('Exact Vertex Cover:',exact)
print('Exact Vertex Cover Size =',len(exact))
print('Approximate Vertex Cover:',approx)
print('Approximate Vertex Cover Size =',len(approx))
print('Approximation Ratio =',round(len(approx)/len(exact),2))
```

Output

```
Exact Vertex Cover: {0, 2, 3}
Exact Vertex Cover Size = 3
Approximate Vertex Cover: {0, 1, 2, 3}
Approximate Vertex Cover Size = 4
Approximation Ratio = 1.33
```

Approximation Algorithm – Bin Packing

Q15. First Fit Decreasing Bin Packing

Hard | Approximation Algorithm

Place items into bins using the First Fit Decreasing strategy.

Input:

Bin Capacity = 10

Items:
[2,5,4,7,1,3,8]

Expected Output:

Sorted Items:
[8,7,5,4,3,2,1]

Bin 1: 8 2
Bin 2: 7 3
Bin 3: 5 4 1

Total Bins Used = 3

main.py

```
capacity=10
items=[2,5,4,7,1,3,8]
sorted_items=sorted(items,reverse=True)
bins=[]
for item in sorted_items:
    for b in bins:
        if sum(b)+item<=capacity:
            b.append(item); break
    else: bins.append([item])
print('Sorted Items:')
print(sorted_items)
for i,b in enumerate(bins,1): print(f'Bin {i}:',*b)
print('Total Bins Used =',len(bins))
```

Output

```
Sorted Items:
[8, 7, 5, 4, 3, 2, 1]
Bin 1: 8 2
Bin 2: 7 3
Bin 3: 5 4 1
Total Bins Used = 3
```

16. N-Queens Using Branch and Bound

Hard | Optimization Technique

Implement the N-Queens problem using the Branch and Bound approach and display one valid arrangement of queens.

Input:

`N = 8`

Expected Output:

```
One Valid Solution:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

main.py

```
def nqueens_branch_bound(n):
    cols=[False]*n; slash=[False]*(2*n-1); back=[False]*(2*n-1); pos=[-1]*n
    def bt(r):
        if r==n:return True
        for c in range(n):
            s=r+c; b=r-c+n-1
            if not cols[c] and not slash[s] and not back[b]:
                pos[r]=c; cols[c]=slash[s]=back[b]=True
                if bt(r+1): return True
                cols[c]=slash[s]=back[b]=False
        return False
    bt(0); return pos
pos=nqueens_branch_bound(8)
print('One Valid Solution:')
for c in pos: print(' '.join('Q' if j==c else '.' for j in range(8)))
```

Output

```
One Valid Solution:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

17. Sudoku Solution Counter

Hard | Backtracking

Determine the total number of valid solutions for a given Sudoku puzzle.

Input:

```
Partially Filled Sudoku Grid
```

Expected Output:

```
Number of Valid Solutions = 1
```

main.py

```python
grid=[
[5,3,0,0,7,0,0,0,0],
[6,0,0,1,9,5,0,0,0],
[0,9,8,0,0,0,0,6,0],
[8,0,0,0,6,0,0,0,3],
[4,0,0,8,0,3,0,0,1],
[7,0,0,0,2,0,0,0,6],
[0,6,0,0,0,0,2,8,0],
[0,0,0,4,1,9,0,0,5],
[0,0,0,0,8,0,0,7,9]]

def safe(g,r,c,num):
    if num in g[r]: return False
    if any(g[i][c]==num for i in range(9)): return False
    br=(r//3)*3; bc=(c//3)*3
    return all(g[i][j]!=num for i in range(br,br+3) for j in range(bc,bc+3))

def solve(g):
    for r in range(9):
        for c in range(9):
            if g[r][c]==0:
                for num in range(1,10):
                    if safe(g,r,c,num):
                        g[r][c]=num
                        if solve(g): return True
                        g[r][c]=0
                return False
    return True

def count_solutions(g,limit=2):
    count=0
    def bt():
        nonlocal count
        if count>=limit:return
        # choose first empty cell
        for r in range(9):
            for c in range(9):
                if g[r][c]==0:
                    for num in range(1,10):
                        if safe(g,r,c,num):
                            g[r][c]=num; bt(); g[r][c]=0
                    return
        count+=1
    bt(); return count
print('Number of Valid Solutions =',count_solutions(grid,10))
```

Output

```
Number of Valid Solutions = 1
```

18. Graph Coloring with Minimum Colors

Medium | Greedy Coloring

Color the vertices of a graph using the Greedy Coloring algorithm and determine the number of colors used.

Input:

```
Vertices = 5

Edges:
(0,1) (0,2) (1,2)
(1,3) (2,4)
```

Expected Output:

```
Vertex 0 → Color 1
Vertex 1 → Color 2
Vertex 2 → Color 3
Vertex 3 → Color 1
Vertex 4 → Color 1

Total Colors Used = 3
```

main.py

```
vertices=5
edges=[(0,1),(0,2),(1,2),(1,3),(2,4)]
g=[set() for _ in range(vertices)]
for u,v in edges:g[u].add(v);g[v].add(u)
color=[0]*vertices
for u in range(vertices):
    used={color[v] for v in g[u] if color[v]}
    c=1
    while c in used:c+=1
    color[u]=c
for v,c in enumerate(color): print(f'Vertex {v} -> Color {c}')
print('Total Colors Used =',max(color))
```

Output

```
Vertex 0 -> Color 1
Vertex 1 -> Color 2
Vertex 2 -> Color 3
Vertex 3 -> Color 1
Vertex 4 -> Color 1
Total Colors Used = 3
```

19. Approximate Vertex Cover Using Edge Picking

Medium | Approximation Algorithm

Find an approximate vertex cover by repeatedly selecting an uncovered edge and adding both its endpoints to the cover.

Input:

```
Vertices = 6

Edges:
(0,1)
(0,2)
(1,3)
(2,4)
(4,5)
```

Expected Output:

```
Approximate Vertex Cover:
{0,1,2,4,5}

Cover Size = 5
```

main.py

```
edges=[(0,1),(0,2),(1,3),(2,4),(4,5)]
remaining=set(edges); cover=set()
while remaining:
    u,v=min(remaining)
    cover.update([u,v])
    remaining={e for e in remaining if u not in e and v not in e}
print('Approximate Vertex Cover:')
print(cover)
print('Cover Size =',len(cover))
```

Output

```
Approximate Vertex Cover:
{0, 1, 2, 4}
Cover Size = 4
```

20. Best Fit Bin Packing

Medium | Approximation Algorithm

Place items into bins using the Best Fit strategy such that each item is assigned to the bin with the least remaining space that can accommodate it.

Input:

```
Bin Capacity = 10

Items:
[2, 5, 4, 7, 1, 3, 8]
```

Expected Output:

```
Bin 1: 2 5 1
Bin 2: 4 3
Bin 3: 7
Bin 4: 8

Total Bins Used = 4
```

main.py

```
capacity=10
items=[2,5,4,7,1,3,8]
bins=[]
for item in items:
    best=-1; best_rem=capacity+1
    for i,b in enumerate(bins):
        rem=capacity-sum(b)
        if item<=rem and rem-item<best_rem:
            best=i; best_rem=rem-item
    if best==-1: bins.append([item])
    else: bins[best].append(item)
for i,b in enumerate(bins,1): print(f'Bin {i}:',*b)
print('Total Bins Used =',len(bins))
```

Output

```
Bin 1: 2 5 1
Bin 2: 4
Bin 3: 7 3
Bin 4: 8
Total Bins Used = 4
```